EnergyPilot: A Proactive Agentic Home Energy Optimizer

Emil Ljungdell
Group EnergyPilot
Hanyang University

Philip Dahlensjö
Group EnergyPilot
Hanyang University

Filip Kadmark
Group EnergyPilot
Hanyang University

Jasmin Hansson
Group EnergyPilot
Hanyang University

TABLE I
ROLE ASSIGNMENTS (TILL THE MIDDLE OF THE TERM)

Roles	Name	Task description
User	Philip Dahlensjö	Sets intents and preferences, tries the prototype and reports usability problems.
Customer	Emil Ljungdell	Represents the household that pays the bill. Prioritizes the requirements and accepts the result.
Software developer	Jasmin Hansson	Designs and implements the agent, the scheduler and the user interface.
Development manager	Filip Kadmark	Tracks deadlines, leads group meetings, follows up tasks and makes sure each deliverable is submitted.

***Abstract*—EnergyPilot is a proposal for a software agent that runs household appliances at the cheapest suitable time, without the resident having to program anything. The resident states an intent, for example "the laundry should be done by 07:00", and the agent turns it into a schedule based on forecast electricity prices. The resident accepts or changes the plan, and every decision comes with a short explanation. This document extends our requirement document with the specification phase. It lists nine requirements in one list, describes a deliberately simple development environment (one language, Python, with a local web interface and a local database, at no cost), reviews the software and algorithms we build on, and specifies at pseudocode level how each requirement will be implemented. The first version uses simulated appliances and synthetic or public data.**

***Index Terms*—smart home, agentic AI, energy optimization, requirements, specification, scheduling**

I. INTRODUCTION

A. Motivation

Electricity prices change during the day, but most households run their appliances when it suits them, which is often when power is expensive. Moving flexible loads such as laundry and dishwashing to cheaper hours lowers the bill and can also ease the peak load on the grid. Doing this by hand takes effort: you need to know the prices, remember to start the machine and be at home when it is time. We think an agent can do this for the resident. It also matches this year's course topic, a proactive agentic home that predicts, decides and acts before a problem occurs.

B. Problem Statement and Client Needs

The client is a household that pays the electricity bill and owns several connected appliances. Its needs are:

- lower electricity cost without losing convenience (the laundry must still be done when the resident needs it),
- no manual programming of schedules or rules,
- full control, so the resident can always change the plan,
- understandable decisions, so the resident can see why the agent did something, and
- careful handling of usage data, since it shows when people are home.

C. Related Software

LG ThinQ lets users control LG appliances remotely, build smart routines and monitor energy use, including a comparison with neighbors and savings goals with notifications [1]. The routines are built by the user, and the energy features mainly report what has already happened.

Samsung SmartThings Energy has an AI Energy Mode that adjusts supported Samsung appliances and TVs to use less energy on selected cycles, and an Away mode for turning devices off when nobody is home [2], [3]. The optimization mostly happens inside each device.

Google Home shows energy history and an energy dashboard for Nest thermostats, with separate heating and cooling totals, and the Nest Learning Thermostat builds a schedule from the household's routine [4], [5]. The energy functions we found are centered on the thermostat.

Matter is a connectivity standard that lets devices from different brands work together [6]. It is relevant if EnergyPilot is later connected to real devices.

In the material we found, none of these products lets the user state an intent (a deadline) that an agent turns into a schedule across several appliances based on electricity prices. We will verify this in the apps before the final report. All online sources in this document were accessed on Nov. 5, 2026.

D. Scope

The first version covers scheduling of flexible appliances (washing machine, dryer, dishwasher and one climate device), price and usage forecasting, a simple user interface and a savings report. The app runs on the user's own computer and keeps its data in a local file, so there is no login in the first

version. Real hardware, billing integration, solar production and a cloud service are out of scope for now.

II. Requirements

The requirements are given in one list. Each is prioritized as Must, Should or Could and can change during the project. Section IV specifies how each requirement is achieved, in the same order.

A. R1: Register appliances (Must)

The user registers an appliance with its name, type, power in watts and typical cycle length. Appliances are simulated at first, behind a small interface with start and stop, so real appliances can be added later.

B. R2: Set an intent (Must)

The user fills in a short form: which appliance, the earliest start and the deadline, for example "done by 07:00". The system shows how it understood the intent before planning. Writing the intent as free text, read by an LLM, is a Could.

C. R3: Electricity prices (Must)

The system has hourly prices for the next 48 hours. We start with a synthetic time-of-use tariff, and public price data can be loaded from a file.

D. R4: Price and usage forecast (Should)

The system predicts hourly prices and household consumption for the coming day, using two methods and keeping the one with the lowest error.

E. R5: Schedule (Must)

For each intent the agent proposes the start time with the lowest cost that meets the deadline, the quiet hours and the maximum household load. The plan is ready in under 10 seconds. When the user accepts it, the simulator starts the appliance at that time.

F. R6: Explanation and decision log (Must)

Every decision is saved with a short explanation, for example "Washer at 02:00, 38 % cheaper than starting at 18:00". The numbers come from the scheduler.

G. R7: Manual override (Must)

The user can cancel a planned run or move it to another time. Manual choices are never changed by the agent.

H. R8: Savings report (Should)

A page shows consumption, cost and the saving compared with a baseline without the agent.

I. R9: Replanning (Could)

When the price forecast changes, the agent can plan the runs that have not started again and tell the user what changed.

TABLE II
Development environment

Item	Choice
PC, Emil Ljungdell	Windows 11 Home 10.0.26200, Intel i5-1135G7 (4 cores), 8 GB RAM
PC, Philip Dahlensjö	**[TODO: OS, CPU, RAM]**
PC, Filip Kadmark	**[TODO: OS, CPU, RAM]**
PC, Jasmin Hansson	**[TODO: OS, CPU, RAM]**
Operating system	Windows, macOS or Linux (same code)
Language	Python 3.12
Interface	Streamlit
Data and ML	pandas, NumPy, scikit-learn
Database	SQLite (one file, built into Python)
Tests	pytest
Version control	Git and GitHub
Documents	LaTeX (IEEEtran), Overleaf
LLM (optional)	LLM API, only to read free-text intents

III. Development Environment

A. Choice of Software Development Platform

Platform. EnergyPilot is a Python application with a web interface that runs on the user's own computer (`streamlit run app.py`) and is opened in a browser. It works the same on Windows, macOS and Linux, and needs no server and no installation for the user. We chose it because we want the simplest setup that every group member can run.

Programming language. Python 3.12 is the only language in the project. It has the data libraries we need (pandas, NumPy, scikit-learn), which our AI course project also uses, and Streamlit lets us build the forms, tables and charts of the interface in the same language. We rejected a separate JavaScript frontend, an API server and a database server, because each would add a second language or one more thing to install and run.

Version control and documents. The code and this document are in a public GitHub repository, https://github.com/PhilipDahlensjo/SE-AI_project. The documents are LaTeX files in the repository (IEEEtran, also usable in Overleaf). We track work with issues and pull requests, and each issue names the requirement it implements.

Development environment. Table II lists what we use. Exact library versions are fixed in `requirements.txt` in the repository.

Cloud (bonus, optional). Running locally is enough for the first version. If we have time at the end, we can deploy the same app on AWS Lightsail, for example the Small-2GB Linux plan (2 vCPU, 2 GB RAM, 12 USD per month) [7].

Cost estimation. All software we use is free or open source and we need no new hardware, so the required cost is 0 USD. Two options are not needed for the first version: deploying on AWS Lightsail for one month (12 USD), and an LLM API for free-text intents, where we would set a spending limit of 10 USD in the provider's console. With both options the total is at most 22 USD (Table III).

TABLE III
COST ESTIMATION

Item	USD
Open-source software, GitHub, Overleaf (free plans)	0
Developer computers (already owned)	0
Required total	**0**
Optional: AWS Lightsail, 1 month	12
Optional: LLM API (spending limit)	≤ 10
Total with options	$\leq$ **22**

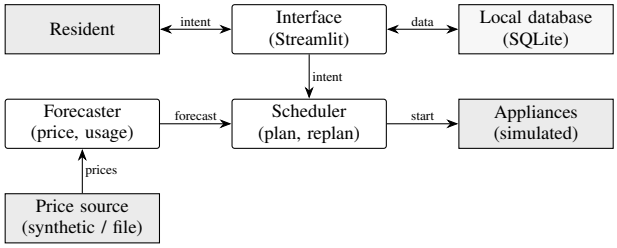


Fig. 1. System overview: main parts and data flows.

B. Software in Use

Products that do a similar task are described in Section I-C. None of them schedules several appliances from a stated intent. We build on the following existing software and algorithms:

- *Seasonal naive forecast*, which predicts the value from 24 hours earlier [8], as the baseline forecaster.
- *Gradient boosting* [9] from scikit-learn [10] as the second forecaster.
- *Exhaustive search* over start slots in the scheduler, which we write ourselves.
- *Streamlit* [11] for the interface and *SQLite* [12] for the data.
- Later, optionally, *LLM tool calling* [13] for free-text intents, and the *LG ThinQ Connect API* [14] and *Matter* [6] for real appliances.

C. Task Distribution

As the assignment allows, the task distribution will be given in the design document (C). The roles until then are in Table I.

IV. SPECIFICATIONS

Fig. 1 shows how the parts fit together, and Table IV lists the main data. Time is divided into 15-minute slots ($\Delta t = 0.25$ h), so 48 hours are 192 slots. Each subsection below belongs to the requirement with the same number in Section II.

A. R1: Register appliances

A device is saved with its data. The simulator implements the same two operations as a real appliance would, so it can be replaced later.

```
procedure REGISTERDEVICE(spec)
    check spec.power > 0 and spec.cycle ≥ 15 min
    return SAVE(Device(spec))
end procedure

interface Appliance: START(), STOP()                ▷ simulated at first
```

TABLE IV
MAIN DATA (ONE SQLITE FILE)

Table	Main fields
Device	id, name, type, power (W), cycle (min)
Setting	quiet hours, maximum household load (W)
Intent	id, device, earliest start, deadline, status
Price	slot start, price per kWh (relative unit in synthetic data)
Consumption	slot start, kWh
Run	id, device, intent, start, cost, baseline cost, status, manual flag
Log	time, run, explanation text

B. R2: Set an intent

The form gives the planner everything it needs. The plan is shown as a proposal, and nothing starts until the user accepts it.

```
procedure SETINTENT(device, earliest, deadline)
    if deadline − earliest < device.cycle then
        show "deadline too close"; return
    end if
    show the intent in plain words
    i ← SAVE(Intent(device, earliest, deadline))
    return PLAN(i)                                  ▷ shown as a proposal
end procedure
```

C. R3: Electricity prices

Prices are read from the database, and missing slots are filled from the synthetic tariff (three price levels over the day plus noise) or from a public price file.

```
procedure GETPRICES(t0)                             ▷ next 48 h
    p ← LOAD(prices from t0)
    if p has gaps then
        fill gaps with SYNTHETICTARIFF(t) (or a price file)
        SAVE(p)
    end if
    return p
end procedure

procedure SYNTHETICTARIFF(t)
    k ← level of hour(t) in {off-peak, mid, peak}
    return base[k] · (1 + ε), ε ~ N(0, σ²)
end procedure
```

D. R4: Price and usage forecast

We compare a baseline and one model, with the last seven days as validation data, and use the one with the lowest mean absolute error. This component is shared with our project in the AI course, which uses public price and consumption data (see the repository).

```
procedure FORECAST(hist)                            ▷ next 24 h
    train ← hist without last 7 days; val ← last 7 days
    for m ∈ {SeasonalNaive, GradientBoosting} do
        fit m on train; features: hour, weekday, y_{t−24}, y_{t−168}
        err[m] ← MAE(m.predict(val), val)
    end for
    m* ← arg min_m err[m]; refit m* on all of hist
    return m*.predict(next 24 h)
end procedure
```

E. R5: Schedule

For one appliance the agent tries every allowed start slot and keeps the cheapest. The cost of a start slot s for an appliance with power P and a cycle of L slots, using price c_t, is

$$C(s) = \sum_{t=s}^{s+L-1} c_t \, P \, \Delta t. \tag{1}$$

Only slots that pass the quiet-hours and load checks are tried, so a plan can never break a hard rule. With 192 slots the search is far below 10 seconds. With several intents, the one with the least slack ($deadline - earliest - L$) is planned first, and each accepted result is added to the load l_t. When the user accepts the proposal, the run is saved and the simulator starts the appliance at that slot.

procedure PLAN(i)
  $d \leftarrow i.device$; $L \leftarrow \lceil d.cycle/15 \rceil$; $P \leftarrow d.power$
  $c \leftarrow$ GETPRICES(now); $l \leftarrow$ load of accepted runs
  $best \leftarrow \perp$; $bestCost \leftarrow \infty$
  for $s \leftarrow i.earliest$ **to** $i.deadline - L$ **do**
    if slots $s..s+L-1$ touch quiet hours **then**
      continue
    end if
    if $\exists t:\ l_t + P > L_{max}$ **then**
      continue
    end if
    $C \leftarrow \sum_{t=s}^{s+L-1} c_t P \Delta t$
    if $C < bestCost$ **then**
      $best \leftarrow s$; $bestCost \leftarrow C$
    end if
  end for
  if $best = \perp$ **then**
    tell the user the intent cannot be met; suggest a later deadline
    return
  end if
  return Run(d, $best$, $bestCost$) ▷ proposal
end procedure

F. R6: Explanation and decision log

The explanation is a text template filled with numbers from the scheduler, so it cannot contain a wrong number.

procedure EXPLAIN(r)
  $base \leftarrow$ cost if started at the earliest allowed time
  $pct \leftarrow 100\,(base - r.cost)/base$
  $text \leftarrow r.device$ at $r.start$, pct % cheaper than $i.earliest$
  SAVE(Log(now, r, $text$))
end procedure

G. R7: Manual override

A run that has already started cannot be changed. A move is checked against the same hard rules as the plan.

procedure OVERRIDE($r, action, t$) ▷ cancel, or move to t
  if r has started **then**
    return error
  end if
  if $action =$ cancel **then**
    $r.status \leftarrow$ cancelled
  else
    if slots from t break a hard rule **then**
      return error
    end if
    $r.start \leftarrow t$
  end if
  $r.manual \leftarrow$ true; LOG(r, "changed by user")
end procedure

H. R8: Savings report

The baseline is a resident without the agent, who starts each run at the earliest time the intent allows. The saving is

$$S = 100\,\frac{C_{base} - C_{agent}}{C_{base}}\ \%. \tag{2}$$

procedure REPORT($from, to$)
  $R \leftarrow$ finished runs in $[from, to]$
  $C_{agent} \leftarrow \sum_{r \in R} r.cost$; $C_{base} \leftarrow \sum_{r \in R} r.baseCost$
  $S \leftarrow 100\,(C_{base} - C_{agent})/C_{base}$
  return daily series (cost, baseline, kWh) and S
end procedure

I. R9: Replanning

Runs that are manual or already started are never touched.

procedure REPLAN ▷ when the price forecast changes
  for each planned run r with **not** $r.manual$, sorted by slack **do**
    $r' \leftarrow$ PLAN($r.intent$) ▷ latest prices
    if $r'.start \neq r.start$ **then**
      $r.start \leftarrow r'.start$; $r.cost \leftarrow r'.cost$
      LOG(r, "replanned")
    end if
  end for
end procedure

REFERENCES

[1] LG Electronics, "LG ThinQ" (app description). [Online]. Available: https://www.apkmirror.com/apk/lgelectronics/lg-smartthinq

[2] SmartThings, "New SmartThings Energy features take home energy management to next level," Samsung Newsroom, Jun. 27, 2023. [Online]. Available: https://news.samsung.com/us/samsung-new-smartthings-energy-features-take-home-energy-management-next-level

[3] Samsung, "How to use SmartThings AI energy saving features." [Online]. Available: https://www.samsung.com/ie/support/apps-services/how-to-use-smartthings-ai-energy-saving-controls/

[4] Google, "Nest Thermostat," Google Store. [Online]. Available: https://store.google.com/product/nest_thermostat?hl=en-us

[5] "Google Home app update brings new features for smart thermostats," Notebookcheck. [Online]. Available: https://www.notebookcheck.net/Google-Home-App-update-brings-new-features-for-smart-thermostats.1098863.0.html

[6] Connectivity Standards Alliance, "Matter." [Online]. Available: https://csa-iot.org/all-solutions/matter/

[7] Amazon Web Services, "Lightsail instance bundles." [Online]. Available: https://docs.aws.amazon.com/lightsail/latest/userguide/amazon-lightsail-bundles.html

[8] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[9] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] Streamlit. [Online]. Available: https://streamlit.io/

[12] SQLite. [Online]. Available: https://www.sqlite.org/

[13] Anthropic, "Claude API documentation." [Online]. Available: https://docs.claude.com

[14] LG Electronics, "ThinQ Connect: Personal Access Token (PAT)." [Online]. Available: https://thinq.developer.lge.com/en/cloud/docs/thinq-connect/PAT-en